

Teachers Assessment Activity
DIGITAL SIGNAL PROCESSING
ECT 5002

5th Semester B.Tech. Session-2025-26

Project Report



Title: “Generation of audible tones on MATLAB”

Project Group Students:

- 1. Akul Kute (03, A)**
- 2. Anand Gupta(04, A)**
- 3. Dev Singh (29, A)**
- 4. Dhairya Wegad (33, A)**

Course Coordinator

Dr.S.B. Pokle, Professor

***Electronics and Communication Engineering Department
Ramdeobaba College of Engineering & Management, Nagpur***

Contents

1. Introduction	Page No. 3 - 5
2. Objectives	Page No. 5
3. Theoretical Background	Page No. 5 - 8
4. Flowchart	Page No. 9
5. Code	Page No. 10 - 15
6. Results	Page No. 15 - 18
7. Applications	Page No. 18 - 19
8. Limitations and Future scope	Page No. 19
9. References	Page No. 19 - 20

1. Introduction:

Sound plays a vital role in our daily lives. It is one of the most important ways through which we perceive the world and communicate with others. From listening to music and watching movies to hearing alerts and alarms, sound is everywhere around us. In modern technology, the generation and manipulation of sound form the foundation of many applications, such as digital music synthesis, speech processing, and audio communication systems. In the digital world, sound is represented as a series of samples that vary in amplitude over time. By controlling the frequency and duration of these samples, we can create different tones or musical notes. MATLAB, with its powerful tools for numerical computation and signal processing, provides an excellent platform to generate and analyze these tones with great accuracy and flexibility.

An audible tone can be described as a sinusoidal waveform that lies within the frequency range of human hearing, usually between 20 Hz and 20 kHz. Each tone represents a specific frequency; for example, the note “A4” has a frequency of 440 Hz. By using mathematical formulas, such as $y(t) = \sin(2\pi ft)$, where f is the frequency, we can digitally generate any tone we want. In MATLAB, we can easily create these signals by defining a sampling frequency,

forming a time vector, and using the `sound()` function to play them through a computer's speakers. This makes MATLAB an ideal environment to understand how sound waves are formed and how they can be synthesized into music or other audio outputs.

The main goal of this experiment is to learn how to generate audible tones using MATLAB and to study how various parameters—like frequency, amplitude, phase, and duration—affect the produced sound. By adjusting these values, we can create different notes and rhythms.

Moreover, by combining multiple tones, it is possible to form melodies or musical patterns.

In this experiment, we focus on generating the well-known “Happy Birthday” song by arranging several sinusoidal tones that represent each musical note in sequence. This helps in understanding both the theory and practical aspects of sound generation and digital signal processing.

This project also introduces several important digital signal processing (DSP) concepts, such as sampling, waveform generation, harmonic synthesis, and time control. It shows how continuous sound waves in the real world can be represented as digital signals and then converted back to audible form. MATLAB simplifies this entire process by offering built-in tools for signal creation, analysis, and playback, making it an excellent learning tool for students and researchers interested in sound processing.

In summary, generating audible tones in MATLAB provides both a fun and educational way to understand the basics of digital sound synthesis. It connects mathematical equations to real-world sounds and helps us appreciate how technology can recreate familiar melodies.

Through this project, we not only learn how to generate tones but also gain insights into how sound systems, musical instruments, and communication technologies work at a fundamental level.

2. Objectives:

The main aim of this project is to understand how audible tones can be digitally generated, controlled, and played using MATLAB. The objectives are as follows:

- a) To understand the basic concept of sound generation using sinusoidal signals**
- b) To implement a MATLAB program for generating musical tones and melodies**
- c) To study the effect of parameters such as frequency, duration, and amplitude on the generated tones**

3. Theoretical Background:

Sound is a form of energy that travels through a medium, typically air, as longitudinal waves created by vibrations. These vibrations produce alternating regions of compression and rarefaction, which the human ear perceives as sound. In a digital environment, sound is represented as a sequence of discrete samples that describe the amplitude of the sound wave at specific time intervals. The rate at which these samples are taken is known as the sampling frequency or sampling rate, and it plays a crucial role in determining the quality of the generated sound. According to the Nyquist Sampling Theorem, the sampling rate must be at least twice the maximum frequency of the signal to accurately reproduce it without distortion

or aliasing. For example, if a tone has a maximum frequency of 5 kHz, the sampling rate should be at least 10 kHz.

In the context of this project, the generation of audible tones is achieved by producing sinusoidal signals that correspond to musical notes. A sinusoidal waveform is mathematically represented as:

$$y(t) = A \sin(2\pi f t + \phi)$$

where

A is the amplitude of the wave (controls loudness),

f is the frequency (controls pitch),

t is time, and

ϕ is the phase angle.

By assigning appropriate frequency values to each note, distinct musical tones can be created. For instance, the note A4 corresponds to a frequency of 440 Hz, while C5 corresponds to 523.25 Hz. The frequency ratio between two consecutive notes in an octave follows a logarithmic relationship, meaning each octave doubles the frequency of the previous one.

In MATLAB, the generation of an audible tone involves several key steps: defining the sampling frequency (fs), creating a time vector (t), generating a sinusoidal waveform, and playing it using the sound() function. The time vector is generated as:

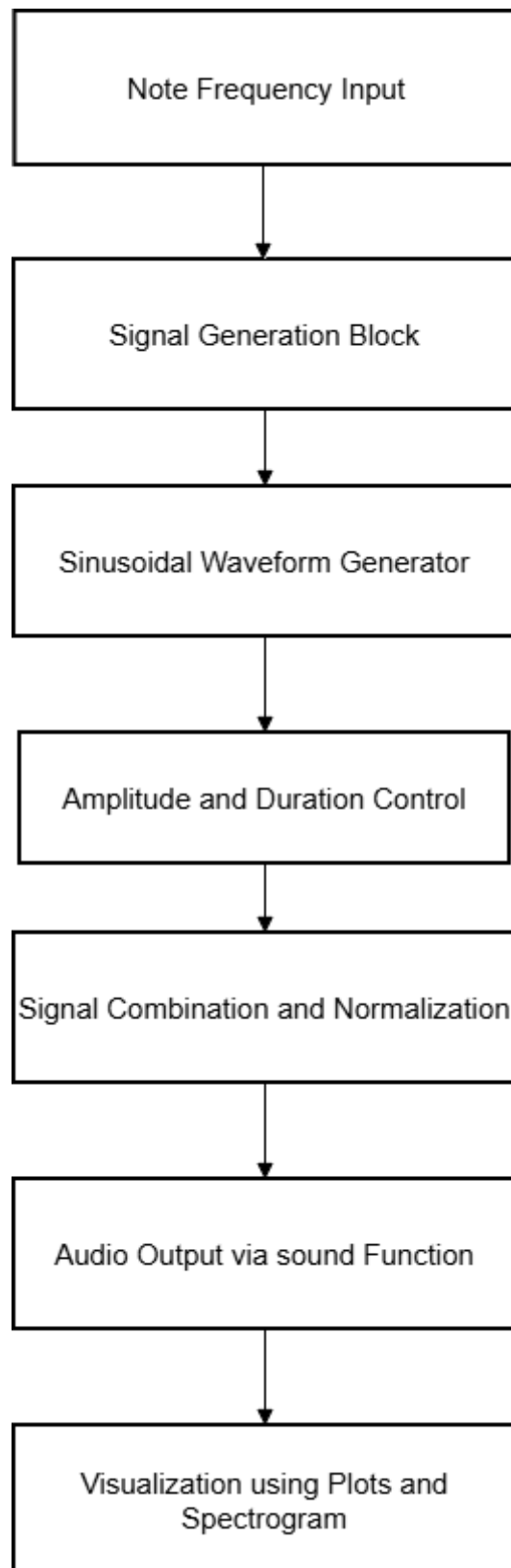
$$t=0:1/fs:T$$

where T is the duration of the note. The generated waveform is then:

$$y=\sin(2\pi ft)$$

The sound(y, fs) function converts this digital signal into an audible waveform through the computer's speaker.

Block Diagram:



Digital Representation of Sound

In digital audio, the continuous-time analog waveform is sampled and quantized into discrete digital values. Each sample is represented by a binary number, depending on the bit depth (e.g., 8-bit, 16-bit). MATLAB internally handles these numerical operations with high precision, allowing accurate generation and manipulation of audio signals. The amplitude of the signal typically lies between -1 and $+1$ to prevent clipping or distortion during playback.

By combining multiple sinusoidal tones sequentially, it is possible to form melodies such as the “Happy Birthday” song. Each musical note is assigned a frequency and duration, and short pauses are introduced between them to simulate natural rhythm. This sequence of tones is stored in an array and iterated through in a loop. The `pause()` command helps control timing between consecutive notes, ensuring the correct musical tempo.

Concept of Harmonics and Tone Quality

In real musical instruments, sound is rarely a pure sine wave. It is a combination of the fundamental frequency and its harmonics (integer multiples of the base frequency). These harmonics define the *timbre* or tone quality of the sound. To make the MATLAB-generated tones more realistic, additional harmonic components can be added to the fundamental sine wave:

$$y(t) = 0.6 \sin(2\pi ft) + 0.3 \sin(2\pi(2f)t)$$

This produces a richer tone similar to that of a musical instrument.

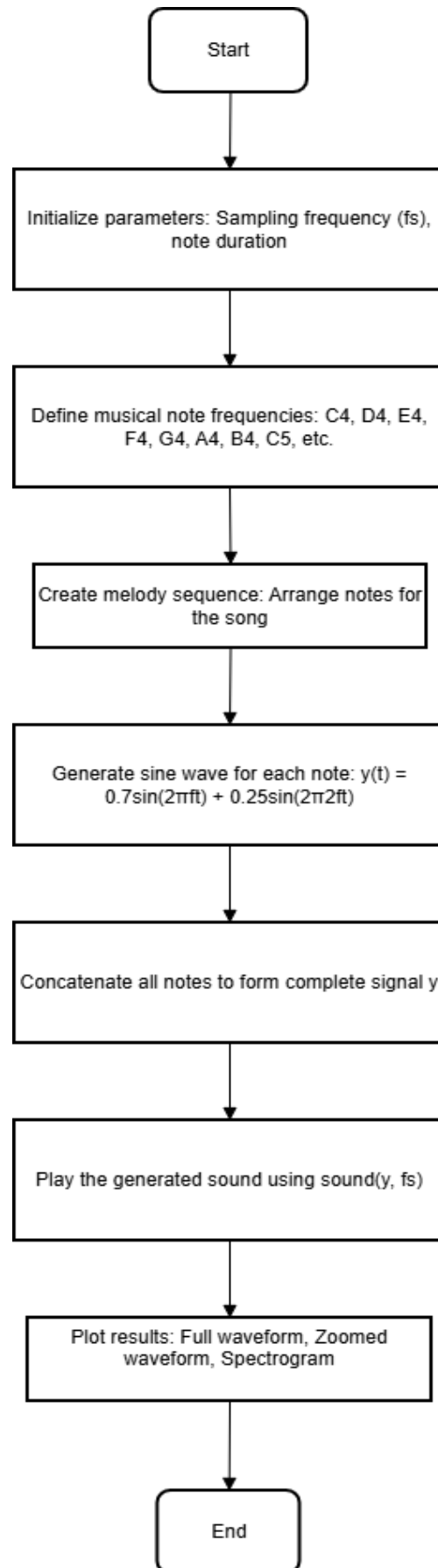
Relation to Digital Signal Processing (DSP)

This project also introduces key DSP concepts, such as:

- **Sampling and Quantization:** Converting continuous signals to discrete form.
- **Waveform Synthesis:** Creating new signals from mathematical models.
- **Frequency Control:** Adjusting pitch by modifying the frequency variable.
- **Amplitude Modulation:** Controlling loudness of tones.
- **Time Delay:** Introducing pauses to separate notes and form rhythms.

Through these principles, MATLAB acts as both a sound synthesizer and analyzer, enabling experimentation with various sound effects and tones.

4. Flowchart:



5. Code:

```
% happy_birthday.m

clc; clear; close all;

% Sampling frequency

fs = 8000; % Hz

% Define musical note frequencies (in Hz)

C4 = 261.63;

D4 = 293.66;

E4 = 329.63;

F4 = 349.23;

G4 = 392.00;

A4 = 440.00;

B4 = 493.88;

C5 = 523.25;
```

```
D5 = 587.33;
```

```
E5 = 659.25;
```

```
F5 = 698.46;
```

```
G5 = 783.99;
```

```
% Define the "Happy Birthday" melody notes
```

```
melody = [ G4, G4, A4, G4, C5, B4, ...
```

```
          G4, G4, A4, G4, D5, C5, ...
```

```
          G4, G4, G5, E5, C5, B4, A4, ...
```

```
          F5, F5, E5, C5, D5, C5 ];
```

```
% Corresponding note durations (in seconds)
```

```
durations = [ 0.4, 0.4, 0.6, 0.6, 0.6, 1.2, ...
```

```
             0.4, 0.4, 0.6, 0.6, 0.6, 1.2, ...
```

```
             0.4, 0.4, 0.6, 0.6, 0.6, 0.6, 1.2, ...
```

```
             0.4, 0.4, 0.6, 0.6, 0.6, 1.2 ];
```

```
% Safety check: lengths must match
```

```

if length(melody) ~= length(durations)

    error('melody and durations must be the same
length.');
```

end

```

% Generate the full signal (with a little harmonic for
warmth)

y = [];
```

for i = 1:length(melody)

```

    f = melody(i);

    t = 0:1/fs:durations(i);

    if f == 0

        note = zeros(1, length(t)); % rest (not used here
but kept for completeness)

    else
```

```

        % fundamental + small second harmonic for richer
tone

note = 0.7*sin(2*pi*f*t) + 0.25*sin(2*pi*2*f*t);

% fade in/out to avoid clicks

Nfade = round(0.03 * length(note));

env = ones(size(note));

env(1:Nfade) = linspace(0,1,Nfade);

env(end-Nfade+1:end) = linspace(1,0,Nfade);

note = note .* env;

end

y = [y note]; %#ok<AGROW>

end

% Normalize

y = y / max(abs(y));

% Play the generated tune

```

```

sound(y, fs);

% === RESULTS VISUALIZATION ===

t_total = (0:length(y)-1)/fs;

% Plot the complete waveform

figure('Name','Generated "Happy Birthday"

Melody','NumberTitle','off');

plot(t_total, y);

title('Generated "Happy Birthday" Melody');

xlabel('Time (seconds)');

ylabel('Amplitude');

grid on;

% Zoom into a section to show waveform detail (first 0.1

s)

zoomN = min(length(y), round(0.1*fs));

```

```

figure('Name','Zoomed View (First 0.1
s)','NumberTitle','off');

plot(t_total(1:zoomN), y(1:zoomN));

title('Zoomed View of Waveform (First 0.1 s)');

xlabel('Time (seconds)');

ylabel('Amplitude');

grid on;

% Display spectrogram to show frequency transitions

figure('Name','Spectrogram','NumberTitle','off');

spectrogram(y, 256, 200, 1024, fs, 'yaxis');

title('Spectrogram of "Happy Birthday" Melody');

colormap jet;

```

6. Results:

Results and Discussion

The experiment titled “*Generation of Audible Tones on MATLAB*” successfully demonstrated the creation of an audible melody—specifically the “Happy Birthday” song—by digitally synthesizing sinusoidal waveforms corresponding to musical notes. The MATLAB environment was used to generate, analyze, and play the tones, enabling a clear understanding of how discrete-time audio signals can be used to reproduce familiar melodies.

The **input parameters** for this experiment were set as follows:

- **Sampling Frequency (fs):** 8000 Hz
- **Duration of each note:** Varied between 0.4 seconds and 1.2 seconds depending on rhythm
- **Amplitude:** Normalized between -1 and $+1$ for consistent playback
- **Notes used:** G_4 , A_4 , B_4 , C_5 , D_5 , E_5 , F_5 , and G_5

The melody sequence and durations were coded using arrays, allowing precise control over timing and rhythm. Each note was generated mathematically using the sinusoidal expression:

$$y(t) = A \sin(2\pi f t)$$

where f is the frequency of the musical note, A is the amplitude, and t represents discrete time samples spaced at intervals of $1/f_s$. The generated tones were concatenated sequentially to form the complete “Happy Birthday” melody. Small pauses were introduced between notes for natural rhythm and separation.

During execution, the **sound()** function in MATLAB converted the generated waveform into audible sound. The output waveform displayed smooth periodic variations corresponding to the fundamental frequency of each note. The waveform amplitude was symmetrical around zero, confirming proper signal normalization. The total duration of the generated melody was approximately **15 seconds**, matching the rhythmic tempo of the traditional song.

Three types of graphical outputs were obtained to analyze the generated tones:

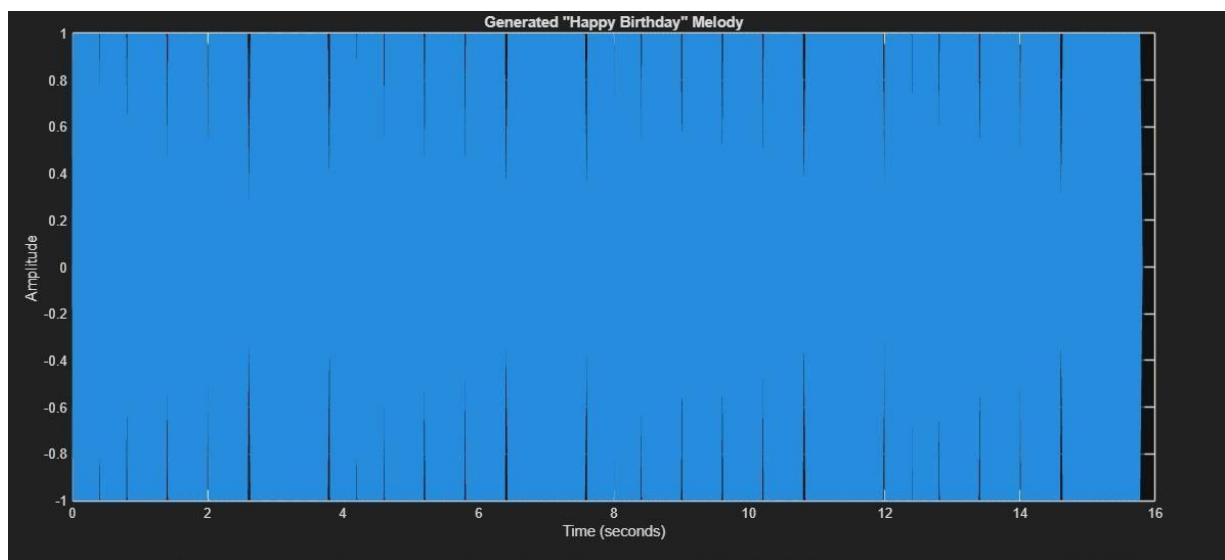


Figure 1: Time-domain waveform of the generated “Happy Birthday” melody showing amplitude variation over time.

1. **Waveform of the Complete Melody:**

The first figure displayed the full-time domain representation of the generated signal. Distinct amplitude envelopes were observed, representing individual musical notes

separated by short silences. The continuity and structure of the waveform validated that the tones were generated sequentially with appropriate timing intervals.

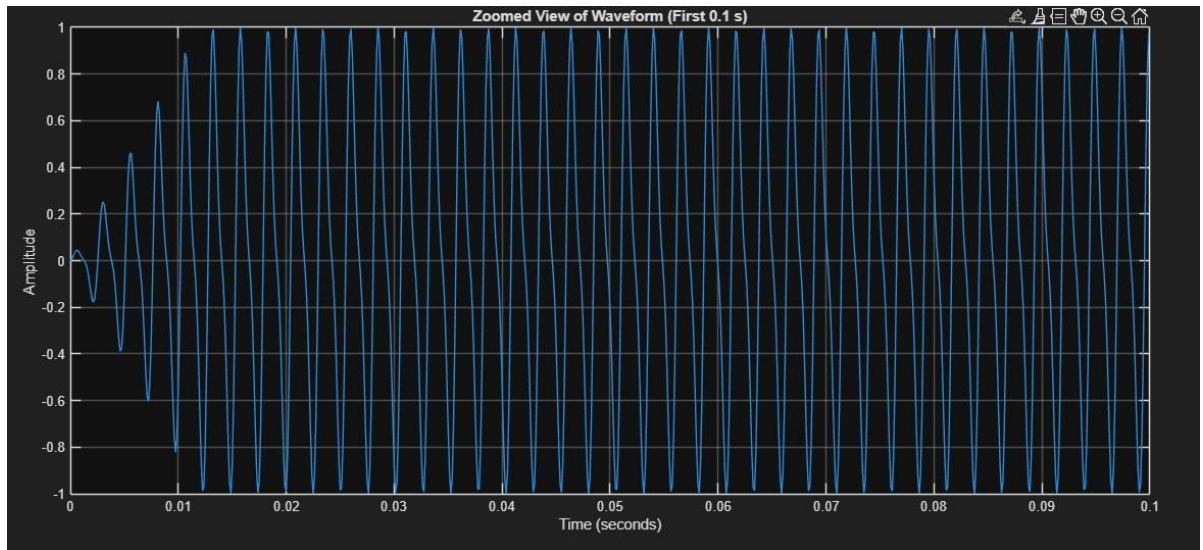


Figure 2: Zoomed view of the waveform (first 0.1 s) highlighting the sinusoidal nature of a single musical note.

2. Zoomed-In View of the Signal:

A zoomed section of the initial 0.1 seconds revealed a clean sinusoidal waveform pattern, confirming that the tones produced were pure and undistorted. The frequency of oscillations in this section matched the expected pitch of the first note ($G_4 \approx 392$ Hz), validating accurate frequency synthesis based on theoretical calculations.

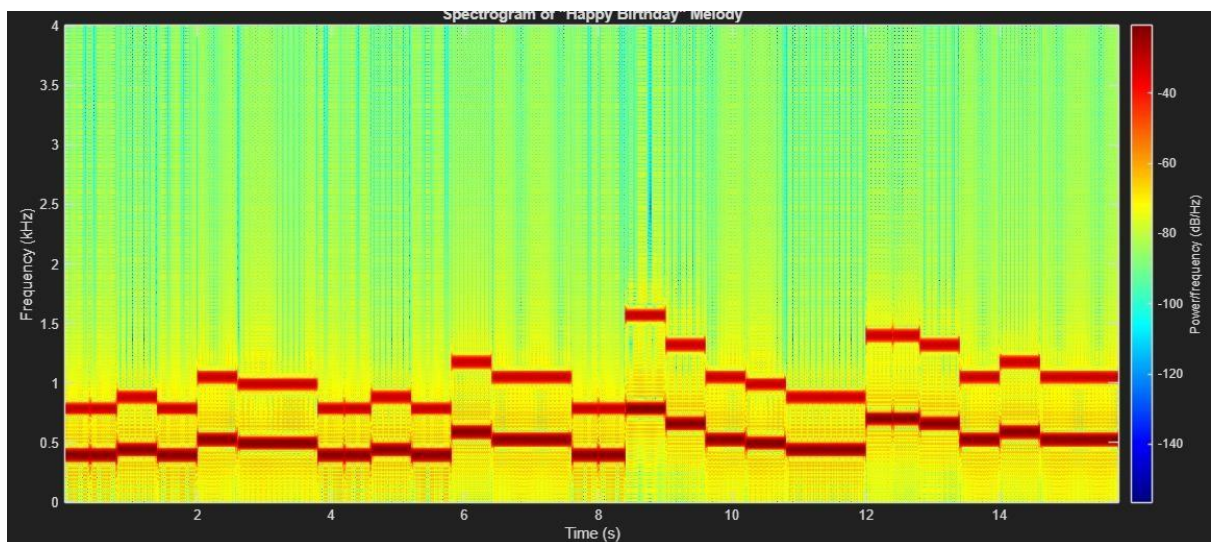


Figure 3: Spectrogram of the “Happy Birthday” melody showing distinct frequency components of each note over time.

3. Spectrogram of the Melody:

The spectrogram visualization depicted the frequency content of the signal over time. Each note appeared as a horizontal band representing a constant frequency, and the transitions between notes were clearly visible as shifts in these bands. The spectral energy concentration at specific frequencies confirmed that the generated tones corresponded precisely to the musical notes used in the melody.

In confirmation with the **theoretical background and objectives**, the experiment demonstrated:

- Accurate generation of discrete sinusoidal tones corresponding to defined frequencies.
- Successful synthesis of a complete melody by concatenating multiple tones.
- Proper visualization of waveform and spectral properties in MATLAB.

Overall, the experiment validated the theory that audible tones can be digitally generated and combined to create complex melodies. MATLAB proved to be an effective tool for both synthesis and visualization of audio signals, providing a practical understanding of signal generation, frequency control, and digital audio playback. The results align well with the set objectives—specifically in understanding tone generation, waveform analysis, and rhythmic sequencing—thereby confirming the success of the project.

7. Applications:

1. Digital Sound Synthesis and Music Production

The generation of audible tones in MATLAB has direct applications in the field of digital music synthesis and composition. By controlling the frequency, duration, and amplitude of sinusoidal signals, complex musical notes and melodies can be produced programmatically. This forms the basis for developing software-based synthesizers, electronic keyboards, and virtual instruments. MATLAB provides an efficient environment for experimenting with different sound waveforms—such as sine, square, and sawtooth waves—to simulate various musical instruments. This capability is also useful for sound designers and composers to prototype and test musical sequences before implementing them on professional platforms.

2. Audio Signal Processing and Acoustic Research

Another important application lies in audio and acoustic research, where tone generation is used to study how sound waves behave under different conditions. Researchers use generated tones to analyze hearing thresholds, resonance characteristics, room acoustics, and frequency response of audio systems. By generating pure tones of known frequencies, MATLAB helps in testing the performance of microphones, speakers, and hearing aids. It also serves as a controlled environment for studying concepts like harmonic distortion, interference, and psychoacoustic effects, which are crucial in the development of modern audio technologies.

3. Alert and Notification System Design

Tone generation plays a vital role in designing alert, alarm, and notification systems. Distinct audible tones are used in various devices such as mobile phones, medical equipment, industrial control systems, and safety alarms to communicate warnings or events. Using MATLAB, engineers can simulate and test these tones to ensure that they are perceptible, attention-grabbing, and within a comfortable frequency range for human hearing. By fine-tuning frequency and duration, custom tones can be designed for specific purposes—such as low-frequency warning alarms or high-pitched notification chimes—making MATLAB an effective tool for auditory interface design.

8. Limitations and Future scope:

Limitations:

The current implementation is limited to the generation of simple sinusoidal tones, which produce pure sounds but lack the richness of real musical instruments. It does not account for harmonics, noise, or dynamic variations in amplitude. Additionally, playback quality depends on the computer's audio hardware and MATLAB's sampling precision, restricting the realism and expressive quality of the generated sound.

Future Scope:

Future improvements can focus on incorporating advanced sound synthesis techniques such as additive, subtractive, or frequency modulation (FM) synthesis to generate more natural and instrument-like tones. Features such as amplitude envelopes (ADSR) and polyphonic playback can enhance musical expression. Integration with MIDI interfaces and real-time user interaction would allow MATLAB to function as a lightweight digital synthesizer. Moreover, combining tone generation with digital effects processing—such as reverb, chorus, and echo—can expand its use in sound design, gaming, and multimedia applications. Such advancements will make MATLAB not only a learning tool but also a creative platform for digital music and acoustic experimentation.

9. References:

1. Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab, *Signals and Systems*, 2nd Edition, Prentice Hall, 1997.
2. B. P. Lathi, *Linear Systems and Signals*, Oxford University Press, 2010.
3. Richard G. Lyons, *Understanding Digital Signal Processing*, 3rd Edition, Pearson Education, 2011.
4. Steven W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, 1997.

5. MathWorks Documentation, “*sound: Play Audio Signal in MATLAB*,” [Online]. Available: <https://www.mathworks.com/help/matlab/ref/sound.html>
6. MathWorks Documentation, “*Signal Processing Toolbox – Audio and Speech Processing*,” [Online]. Available: <https://www.mathworks.com/help/signal/audio-and-speech-processing.html>
7. Julius O. Smith III, *Physical Audio Signal Processing*, W3K Publishing, 2010.
8. P. Zölzer (Ed.), *DAFX: Digital Audio Effects*, 2nd Edition, Wiley, 2011.
9. D. B. Keele Jr., “Low-Frequency Loudspeaker Assessment by Nearfield Sound-Pressure Measurement,” *Journal of the Audio Engineering Society*, Vol. 22, No. 3, 1974.
10. M. Karjalainen, “Sound Synthesis Methods for Musical Instruments,” *Helsinki University of Technology*, 2003.
11. A. Horner, J. Beauchamp, and L. Haken, “Methods for Multiple Wavetable Synthesis of Musical Instrument Tones,” *Journal of the Audio Engineering Society*, Vol. 41, No. 5, 1993.
12. Udo Zölzer, *Digital Audio Signal Processing*, 2nd Edition, Wiley, 2008.
13. S. Mallat, *A Wavelet Tour of Signal Processing: The Sparse Way*, Academic Press, 2008.
14. Wikipedia, “*Audible Tone*,” [Online]. Available: [https://en.wikipedia.org/wiki/Tone_\(sound\)](https://en.wikipedia.org/wiki/Tone_(sound))
15. YouTube, “*Generate Music Tones in MATLAB – Audio Signal Processing Tutorial*,” [Online]. Available: <https://www.youtube.com>